

Indian Institute of Technology, Jodhpur
2025-26

Resilient State Machine Replication using Paxos and PBFT

Fundamentals of Distributed Systems
Assignment 1

Submitted By: Shifali Chandra
Roll Number: G25AI1040

1. Objective

The objective of this assignment is to design and implement a resilient distributed transaction ledger capable of handling both crash faults and Byzantine faults.

The system integrates:

- Leader Election
- Process Coordination
- Paxos Consensus
- Practical Byzantine Fault Tolerance (PBFT)
- Digital Signatures
- Byzantine Fault Simulation
- Docker-based Deployment
- Chaos Testing

The implementation consists of a cluster of five distributed nodes that maintain a replicated transaction ledger while preserving consistency and fault tolerance.

2. System Architecture

Components

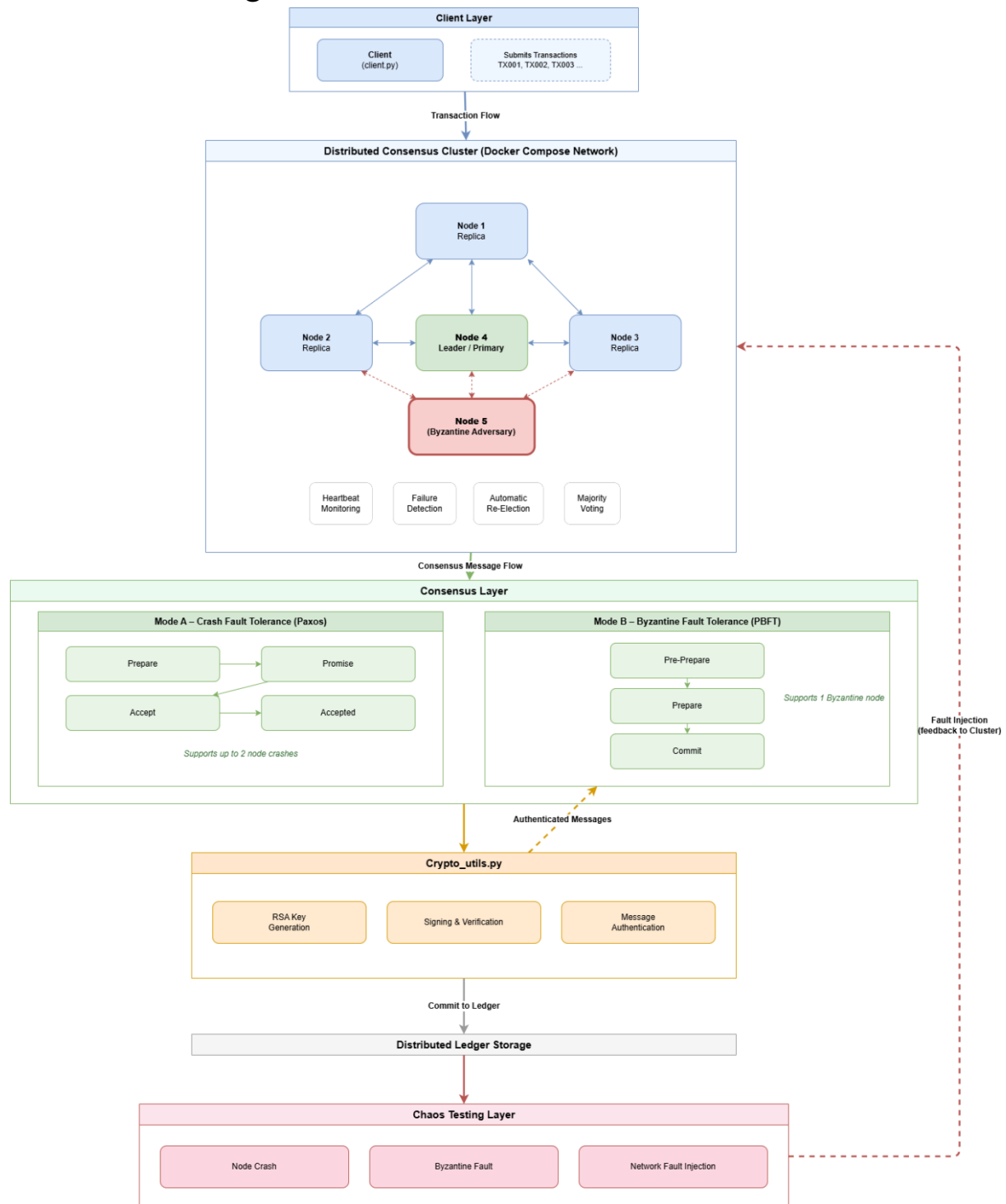
The system consists of:

1. Client Simulator
2. Five Distributed Nodes
3. Leader Election Module
4. Paxos Consensus Engine
5. PBFT Consensus Engine
6. Cryptographic Signature Module
7. Byzantine Adversary Node
8. Docker-Based Cluster Deployment
9. Chaos Testing Framework

Architecture Overview

The system follows a replicated state machine architecture consisting of five distributed nodes and one client. Client transactions are submitted to the cluster leader, which coordinates consensus among replicas. In Crash Fault Tolerance mode, the leader acts as the Paxos proposer and drives the Prepare, Promise, Accept, and Accepted phases. In Byzantine Fault Tolerance mode, the leader acts as the PBFT primary and coordinates the Pre-Prepare, Prepare, and Commit phases. Committed transactions are written to node-specific ledger files only after consensus is achieved. Docker Compose provides cluster orchestration, while chaos testing validates fault recovery under node failures and network disruptions.

Architecture Diagram



3. Process Coordination and Leader Election

Design

A heartbeat-based leader election mechanism is implemented.

Each node operates in one of three states:

- Follower
- Candidate
- Leader

The leader periodically broadcasts heartbeat messages to all followers.

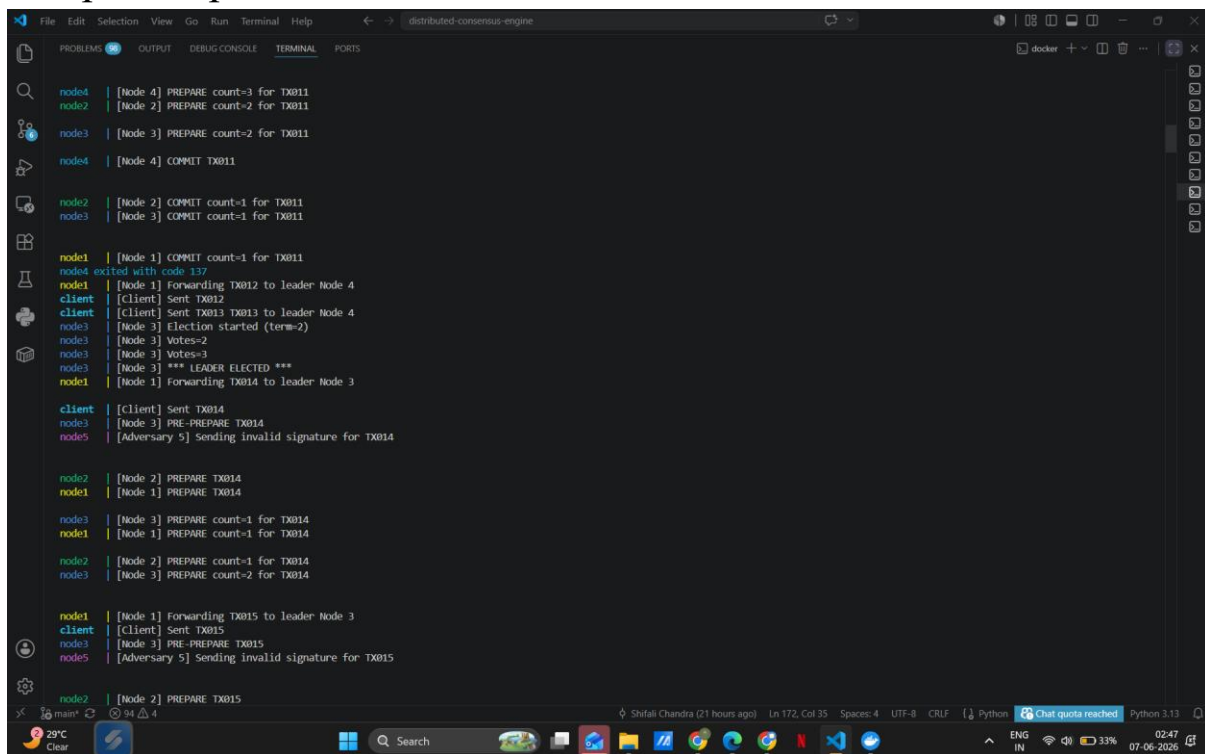
If a follower does not receive a heartbeat within the election timeout period:

1. It becomes a candidate.
2. Starts a new election term.
3. Requests votes from peers.
4. Becomes leader after receiving majority votes.

The elected leader acts as:

- Proposer in Paxos Mode
- Primary in PBFT Mode

Sample Output



```
node4 | [Node 4] PREPARE count=3 for TX011
node2 | [Node 2] PREPARE count=2 for TX011
node3 | [Node 3] PREPARE count=2 for TX011
node4 | [Node 4] COMMIT TX011
node2 | [Node 2] COMMIT count=1 for TX011
node3 | [Node 3] COMMIT count=1 for TX011
node1 | [Node 1] COMMIT count=1 for TX011
node4 exited with code 137
node1 | [Node 1] Forwarding TX012 to leader Node 4
client | [Client] Sent TX012
client | [Client] Sent TX013 TX013 to leader Node 4
node3 | [Node 3] Election started (term=2)
node3 | [Node 3] Votes=2
node3 | [Node 3] Votes=3
node3 | [Node 3] *** LEADER ELECTED ***
node1 | [Node 1] Forwarding TX014 to leader Node 3
client | [Client] Sent TX014
node3 | [Node 3] PRE-PREPARE TX014
node5 | [Adversary 5] Sending invalid signature for TX014
node2 | [Node 2] PREPARE TX014
node1 | [Node 1] PREPARE TX014
node3 | [Node 3] PREPARE count=1 for TX014
node1 | [Node 1] PREPARE count=1 for TX014
node2 | [Node 2] PREPARE count=1 for TX014
node3 | [Node 3] PREPARE count=2 for TX014
node1 | [Node 1] Forwarding TX015 to leader Node 3
client | [Client] Sent TX015
node3 | [Node 3] PRE-PREPARE TX015
node5 | [Adversary 5] Sending invalid signature for TX015
node2 | [Node 2] PREPARE TX015
```

Example:

Node 1 Election started (term=1)

Votes=2

Votes=3

*** LEADER ELECTED ***

Node State Transitions

Each node operates in one of three states:

- Follower → Candidate → Leader

A node begins as a follower. If heartbeat messages from the leader are not received within the election timeout period, the node transitions to the Candidate state and initiates an election. Upon receiving votes from a majority of nodes, the candidate transitions to the Leader state and begins broadcasting heartbeats. If a leader fails, another follower automatically transitions to candidate and the election process repeats.

4. Paxos Consensus Implementation

Overview

Crash fault tolerance is implemented using Basic Paxos.

Consensus proceeds through the following phases:

Phase 1: Prepare

Leader sends PREPARE request.

Phase 2: Promise

Acceptors reply with PROMISE messages.

Phase 3: Accept

Leader sends ACCEPT request after majority promises.

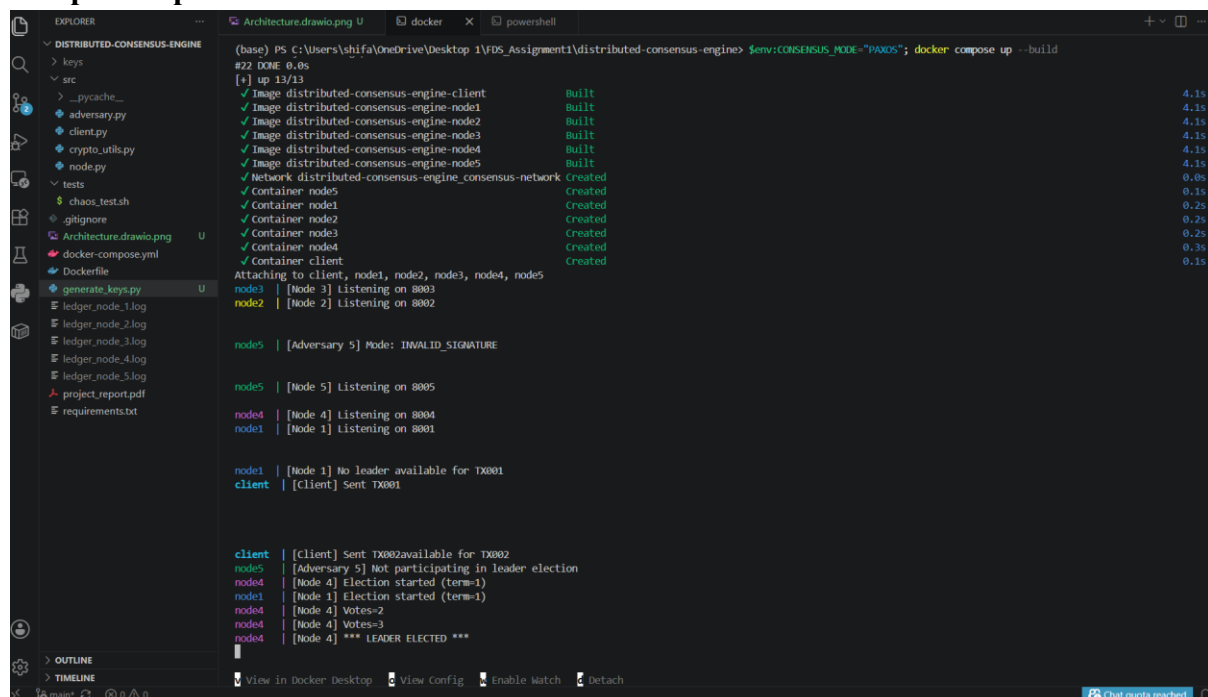
Phase 4: Accepted

Acceptors send ACCEPTED messages.

Commit

Transaction is committed after majority acceptance.

Sample Output



```
(base) PS C:\Users\shifa\OneDrive\Desktop\1\FDS_Assignment1\distributed-consensus-engine> $env:CONSENSUS_MODE="PAXOS"; docker compose up --build
#22 DONE 0.0s
[*] up 13/13
  Image distributed-consensus-engine-client      Built      4.1s
  Image distributed-consensus-engine-node1      Built      4.1s
  Image distributed-consensus-engine-node2      Built      4.1s
  Image distributed-consensus-engine-node3      Built      4.1s
  Image distributed-consensus-engine-node4      Built      4.1s
  Image distributed-consensus-engine-node5      Built      4.1s
  Network distributed-consensus-engine_network  Created    0.1s
  Container nodes                               Created    0.2s
  Container node1                               Created    0.2s
  Container node2                               Created    0.2s
  Container node3                               Created    0.2s
  Container node4                               Created    0.3s
  Container client                              Created    0.1s
Attaching to client, node1, node2, node3, node4, node5
node3 | [Node 3] Listening on 8003
node2 | [Node 2] Listening on 8002
node5 | [Adversary 5] Mode: INVALID_SIGNATURE
node5 | [Node 5] Listening on 8005
node4 | [Node 4] Listening on 8004
node1 | [Node 1] Listening on 8001
node1 | [Node 1] No leader available for TX001
client | [Client] Sent TX001
client | [Client] Sent TX002available for TX002
node5 | [Adversary 5] Not participating in leader election
node4 | [Node 4] Election started (term=1)
node1 | [Node 1] Election started (term=1)
node4 | [Node 4] Votes=2
node4 | [Node 4] Votes=3
node4 | [Node 4] *** LEADER ELECTED ***
```

Fault Tolerance

The Paxos implementation tolerates crash failures while maintaining transaction consistency.

On stopping leader → node4 -

```
(base) PS C:\Users\shifa\OneDrive\Desktop\1\FDS_Assignment1\distributed-consensus-engine> $env:CONSENSUS_MODE="PAXOS"; docker compose up --build

node1 | [Node 1] Sending PROMISE for TX016
node4 | [Node 4] PROMISE count=3
node4 | [Node 4] ACCEPT TX016new Config | Enable Watch | Detach
node1 | [Node 1] Sending ACCEPTED for TX016
node4 | [Node 4] ACCEPTED count=3
node4 | [Node 4] Transaction committed: TX016
node4 | [Node 4] Paxos consensus reached for TX016
node4 exited with code 137

client | [Client] Sent TX017
node1 | [Node 1] Forwarding TX017 to leader Node 4

node5 | [Adversary 5] Not participating in leader election

client | [Client] Sent TX018
node1 | [Node 1] Forwarding TX018 to leader Node 4

node3 | [Node 3] Election started (term=2)
node5 | [Adversary 5] Not participating in leader election
node2 | [Node 2] Election started (term=2)

node3 | [Node 3] Votes=2
node3 | [Node 3] Votes=3
node3 | [Node 3] *** LEADER ELECTED ***

View in Docker Desktop | View Config | Enable Watch | Detach
```

On stopping new leader → node 3 -

```
(base) PS C:\Users\shifa\OneDrive\Desktop\1\FDS_Assignment1\distributed-consensus-engine> $env:CONSENSUS_MODE="PAXOS"; docker compose up --build

node3 | [Node 3] PROMISE count=3
node3 | [Node 3] ACCEPT TX030
node3 | [Node 3] ACCEPTED count=2
node3 | [Node 3] ACCEPTED count=3
node3 | [Node 3] Transaction committed: TX030
node3 | [Node 3] Paxos consensus reached for TX030
node5 | [Adversary 5] Not participating in leader election
node3 exited with code 137

node2 | [Node 2] Election started (term=3)
node5 | [Adversary 5] Not participating in leader election
node2 | [Node 2] Votes=2
node2 | [Node 2] Votes=3
node2 | [Node 2] *** LEADER ELECTED ***

node1 | [Node 1] Forwarding TX031 to leader Node 3

client | [Client] Sent TX031View Config | Enable Watch | Detach

node5 | [Node 5] Sending PROMISE for TX032

node5 | [Node 5] Sending ACCEPTED for TX032
node1 | [Node 1] Sending PROMISE for TX032

node2 | [Node 2] PROMISE count=2
node1 | [Node 1] Sending ACCEPTED for TX032

node2 | [Node 2] PROMISE count=3
node2 | [Node 2] ACCEPT TX032
node2 | [Node 2] ACCEPTED count=2
node2 | [Node 2] ACCEPTED count=3
node2 | [Node 2] Transaction committed: TX032
```

Thus, this shows two nodes have been stopped to simulate crash failures. The remaining majority continues processing transactions successfully.

5. PBFT Consensus Implementation

Overview

PBFT mode is used to tolerate Byzantine faults.

The protocol follows three phases:

Phase 1: Pre-Prepare

Primary proposes a transaction.

Phase 2: Prepare

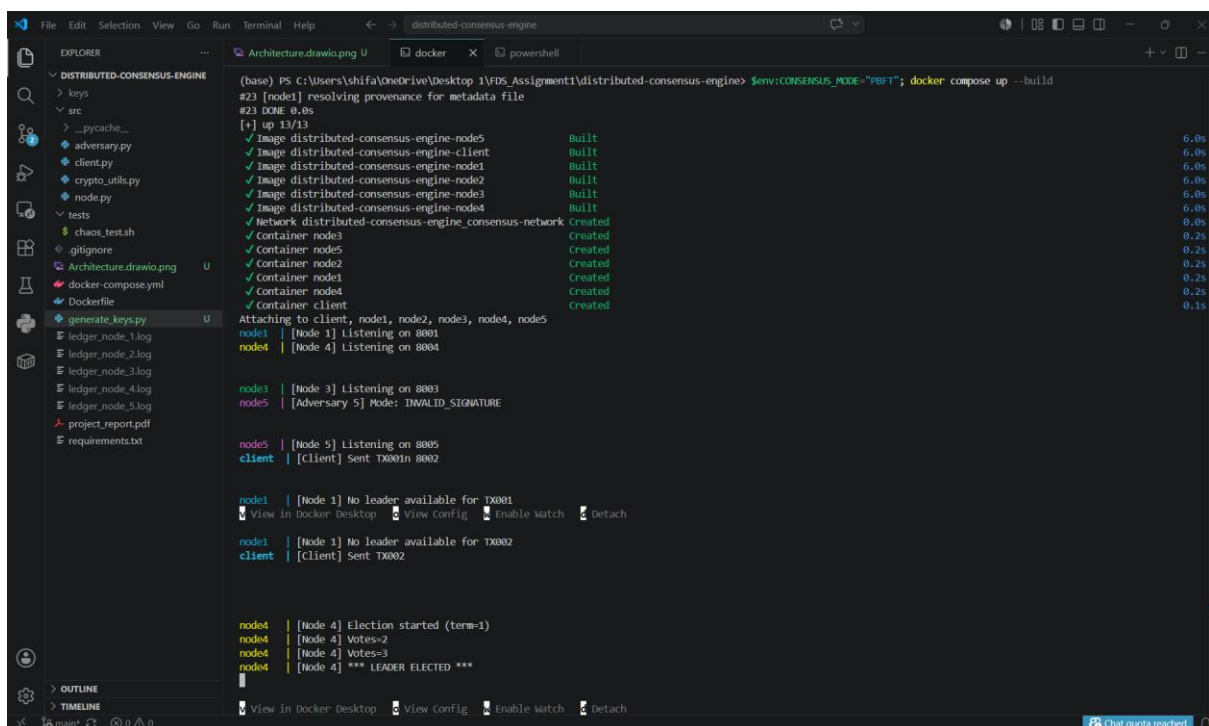
Replicas validate and broadcast prepare messages.

Phase 3: Commit

Nodes exchange commit messages.

Transaction is committed after receiving the required quorum.

Sample Output



```
(base) PS C:\Users\shifa\OneDrive\Desktop\1\FDS_Assignment1\distributed-consensus-engine> $env:CONSENSUS_MODE="PBFT"; docker compose up --build
#23 [node1] resolving provenance for metadata file
#23 DONE 0.0s
[+] up 13/13
✓ Image distributed-consensus-engine-node5 Built 6.0s
✓ Image distributed-consensus-engine-client Built 6.0s
✓ Image distributed-consensus-engine-node1 Built 6.0s
✓ Image distributed-consensus-engine-node2 Built 6.0s
✓ Image distributed-consensus-engine-node3 Built 6.0s
✓ Image distributed-consensus-engine-node4 Built 6.0s
✓ Network distributed-consensus-engine_consensus-network Created 0.0s
✓ Container node3 Created 0.2s
✓ Container node5 Created 0.2s
✓ Container node2 Created 0.2s
✓ Container node1 Created 0.2s
✓ Container node4 Created 0.2s
✓ Container client Created 0.1s
Attaching to client, node1, node2, node3, node4, node5
node1 | [Node 1] Listening on 8001
node4 | [Node 4] Listening on 8004

node3 | [Node 3] Listening on 8003
node5 | [Adversary 5] Mode: INVALID_SIGNATURE

node5 | [Node 5] Listening on 8005
client | [Client] Sent TX001n 8002

node1 | [Node 1] No leader available for TX001
node1 | View in Docker Desktop View Config Enable Watch Detach

node1 | [Node 1] No leader available for TX002
client | [Client] Sent TX002

node4 | [Node 4] Election started (term=1)
node4 | [Node 4] Votes=2
node4 | [Node 4] Votes=3
node4 | [Node 4] *** LEADER ELECTED ***

node1 | View in Docker Desktop View Config Enable Watch Detach
```

Example:

PRE-PREPARE TX001

PREPARE count=1

PREPARE count=2

PREPARE count=3

COMMIT TX001

COMMIT count=1

COMMIT count=2

COMMIT count=3

Transaction committed: TX001

PBFT consensus reached for TX001

Byzantine Fault Tolerance

The system continues operation despite one malicious node in a five-node cluster.

6. Cryptographic Security

Digital Signatures

RSA public-private key pairs are generated for each node.

Each consensus message is:

1. Digitally signed by sender.
2. Verified by receiver.

Security Benefits

- Authentication
- Integrity
- Non-repudiation

Invalid signatures are rejected before participation in consensus.

Key Generation

Keys are generated using:

`generate_key_pair(node_id)`

Each node stores:

- `node_X_private.pem`
- `node_X_public.pem`

Key Distribution

Each node generates an RSA public-private key pair during initialization. Public keys are shared with all participating replicas and are used to verify signatures received from peers. Private keys never leave the originating node and are used only for signing outgoing consensus messages. This mechanism prevents message spoofing and ensures that PBFT replicas can verify message authenticity before participating in consensus.

7. Byzantine Adversary Simulation

Objective

To simulate malicious behavior in PBFT mode.

A dedicated adversary node is implemented.

Supported Attack Modes

Invalid Signature Attack

Node sends forged signatures.

Example:

[Adversary 5] Sending invalid signature for TX001

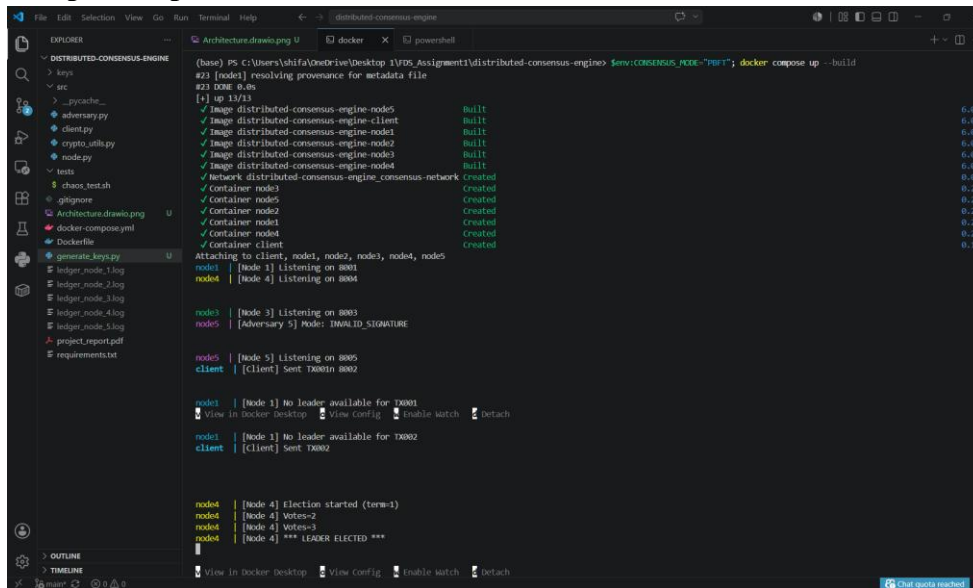
Drop Commit Attack

Node intentionally drops commit messages.

Equivocation Attack

Node sends conflicting transaction information to different peers.

Sample Output



```
(base) PS C:\Users\shifa\OneDrive\Desktop\1VDP_Assignment1\distributed-consensus-engine $env:CONSENSUS_MODE="PHT"; docker compose up --build
#23 [node1] resolving provenance for metadata file
#23 DONE 0.0s
[+] up 13/13
✓ Image distributed-consensus-engine-node5      Built        6.4s
✓ Image distributed-consensus-engine-client      Built        6.4s
✓ Image distributed-consensus-engine-node1       Built        6.4s
✓ Image distributed-consensus-engine-node2       Built        6.4s
✓ Image distributed-consensus-engine-node3       Built        6.4s
✓ Image distributed-consensus-engine-node4       Built        6.4s
✓ Network distributed-consensus-engine_consensus-network Created      0.2s
✓ Container node5                               Created      0.2s
✓ Container node2                               Created      0.2s
✓ Container node1                               Created      0.2s
✓ Container node4                               Created      0.2s
✓ Container client                              Created      0.1s
Attaching to client, node1, node2, node3, node4, nodes
node1 | [Node 1] listening on 8001
node4 | [Node 4] listening on 8004
node3 | [Node 3] listening on 8003
node5 | [Adversary 5] Node: INVALID_SIGNATURE
nodes | [Node 5] listening on 8005
client | [Client] Sent TX001in 8002
node1 | [Node 1] No leader available for TX001
client | [Client] Sent TX002
node1 | [Node 1] No leader available for TX002
client | [Client] Sent TX002
node4 | [Node 4] Election started (term=1)
node4 | [Node 4] Votes=2
node4 | [Node 4] Votes=3
node4 | [Node 4] *** LEADER ELECTED ***
```

The honest nodes continue processing transactions despite malicious behavior.

8. Docker-Based Deployment

Dockerfile

A Docker image is created for all nodes.

The image includes:

- Python Runtime
- Dependencies
- Consensus Engine
- Client Simulator

Docker Compose

Docker Compose is used to deploy:

- Node 1
- Node 2
- Node 3
- Node 4
- Node 5 (Byzantine Node)
- Client Container

Benefits

- Reproducibility
- Isolation
- Simplified Deployment
- Multi-node Simulation

9. Chaos Testing

Objective

To validate fault recovery mechanisms.

A chaos testing script is implemented.

Scenario 1

Stop Leader Node

Expected Result:

- Followers detect failure.
- New election occurs.
- New leader elected.

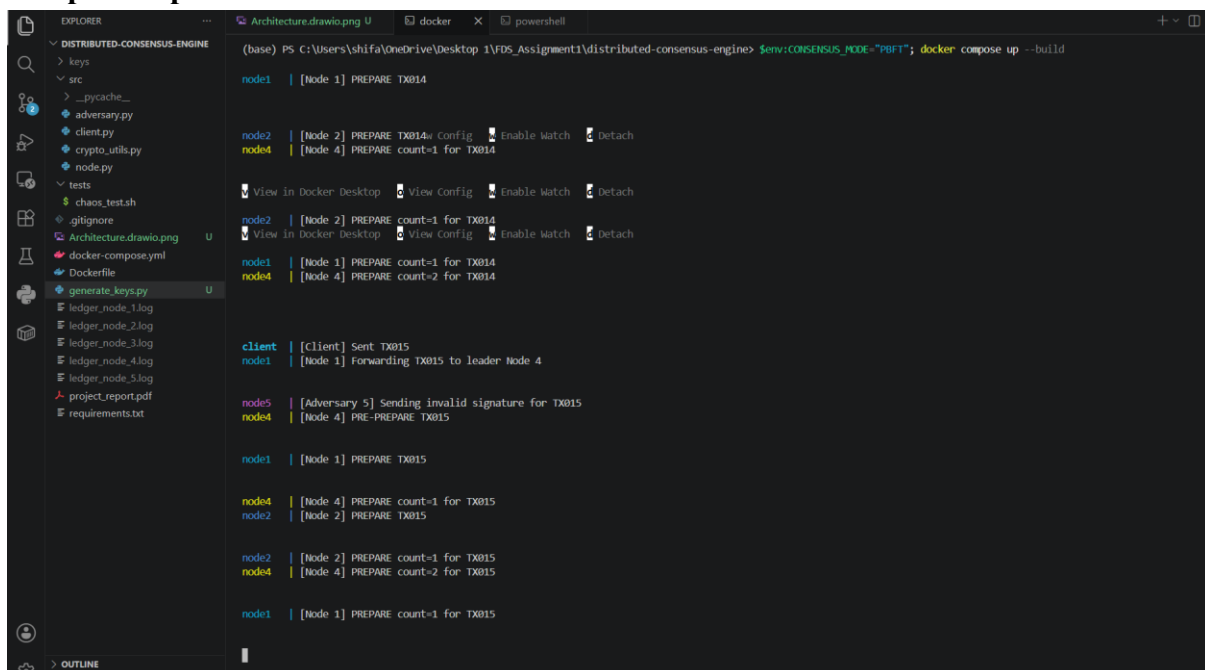
Scenario 2

Restart Failed Node

Expected Result:

- Node rejoins cluster.
- Consensus continues.

Sample Output



```
(base) PS C:\Users\shifa\OneDrive\Desktop\1\FDS_Assignment1\distributed-consensus-engine> $env:CONSENSUS_MODE="PBFT"; docker compose up --build

node1 | [Node 1] PREPARE TX014

node2 | [Node 2] PREPARE TX014w Config | Enable Watch | Detach
node4 | [Node 4] PREPARE count=1 for TX014

View in Docker Desktop | View Config | Enable Watch | Detach

node2 | [Node 2] PREPARE count=1 for TX014
View in Docker Desktop | View Config | Enable Watch | Detach

node1 | [Node 1] PREPARE count=1 for TX014
node4 | [Node 4] PREPARE count=2 for TX014

client | [Client] Sent TX015
node1 | [Node 1] Forwarding TX015 to leader Node 4

node5 | [Adversary 5] Sending invalid signature for TX015
node4 | [Node 4] PRE-prepare TX015

node1 | [Node 1] PREPARE TX015

node4 | [Node 4] PREPARE count=1 for TX015
node2 | [Node 2] PREPARE TX015

node2 | [Node 2] PREPARE count=1 for TX015
node4 | [Node 4] PREPARE count=2 for TX015

node1 | [Node 1] PREPARE count=1 for TX015
```

Example: Stopping node3

Election started (term=3)

Votes=2

Votes=3

*** LEADER ELECTED ***

Consensus continues successfully.

Chaos Evaluation Result

During testing, the active leader node was intentionally terminated using Docker. The remaining replicas detected the failure through missed heartbeat messages and initiated a new election. A replacement leader was elected using majority voting, after which transaction processing resumed successfully. This demonstrates that the cluster can recover from node failures without manual intervention while preserving consensus and service availability.

10. Results

Leader Election

Successfully implemented and validated.

Paxos

Prepare, Promise, Accept, and Accepted phases executed successfully.

PBFT

Pre-Prepare, Prepare, and Commit phases executed successfully.

Byzantine Fault Tolerance

System remained operational despite malicious node behavior.

Docker Deployment

Successfully deployed and tested using Docker Compose.

Chaos Testing

Leader failures were recovered automatically through re-election.

11. Challenges Faced

- Preventing multiple simultaneous leaders.
- Synchronizing asynchronous consensus messages.
- Managing quorum calculations.
- Handling Byzantine behavior correctly.
- Docker networking configuration.
- Ensuring successful leader re-election after failures.

12. Conclusion

A resilient distributed transaction ledger was successfully implemented using Python, asyncio, Docker, Paxos, and PBFT.

The system demonstrates:

- Reliable leader election
- Crash fault tolerance through Paxos
- Byzantine fault tolerance through PBFT
- Secure communication using digital signatures
- Recovery from node failures
- Containerized deployment using Docker

The final implementation satisfies the requirements of the assignment and demonstrates practical distributed consensus under both crash-fault and Byzantine-fault environments.

13. GitHub Repository

Repository Link: <https://github.com/Shifali-Chandra/distributed-consensus-engine>

14. Video Demonstration

Video Link:

<https://drive.google.com/file/d/1k3shgJbz4WeKPjIpdRL3mN3jwdylZgWw/view?usp=sharing>